

Artificial Intelligence - Report Lab 3

Anh Khoi Pham - ITCSIU23018

April 16, 2026

1 The Algorithm

Heuristic Function

The heuristic function $h(n)$ is the estimated cost of the cheapest path from node n to a goal node and it is defined by the Manhattan Distance formula

$$h(n) = |n_x - goal_x| + |n_y - goal_y|$$

```
1 def heuristic(self, a, b):
2     return abs(a[0] - b[0]) + abs(a[1] - b[1])
```

Best-First Search

```
1 def best_fs(self, return_home=False):
2     start = (self.state.pos_x, self.state.pos_y)
3     queue = []
4     explored = set()
5
6     heapq.heappush(queue, (0, start, []))
7
8     while queue:
9         _, node, path = heapq.heappop(queue)
10
11         if node in explored:
12             continue
13         explored.add(node)
14
15         x, y = node
16
17         if return_home:
18             if node == self.home_pos:
19                 self.route = path.copy()
20                 return
21         else:
22             if self.state.world[x][y] ==
23                 AGENT_STATE_UNKNOWN:
24                 self.route = path.copy()
25                 return
```

```
26         for dx, dy in [(1, 0), (0, 1), (-1, 0),
27                        (0, -1)]:
28             nx, ny = x + dx, y + dy
29
30             # check boundary
31             if not (0 <= nx < self.state.
32                    world_width and 0 <= ny < self.
33                    state.world_height):
34                 continue
35
36             # check wall
37             if self.state.world[nx][ny] ==
38                 AGENT_STATE_WALL:
39                 continue
40
41             child = (nx, ny)
42             if child in explored or any(child ==
43                                         n for _, n, _ in queue):
44                 continue
45
46             priority = self.heuristic(child, self
47                                     .home_pos if return_home else (nx,
48                                     ny))
49             heapq.heappush(queue, (priority,
50                                   child, path + [child]))
51
52         self.node_explored = len(explored)
```

How it works? The algorithm works as follows. While the `queue` is not empty, it pop out the first node together with the path leading to it. Next, it checks whether the map has already been fully explored and whether the agent is returning home. If the current node is still marked as `UNKNOWN`, the agent still has unexplored space to visit, so the route is updated with the path to that node. Otherwise, the algorithm expands the current node by adding its valid children to the `queue` based on their heuristic score (in this case, it is the `priority` variable).

A* Algorithm

```
1 def a_star(self, return_home=False):
2     start = (self.state.pos_x, self.state.pos_y)
```

```
3     queue = []
4     explored = set()
5     g_scores = {start: 0}
6
7     heapq.heappush(queue, (0, start, []))
8
9     while queue:
10         _, node, path = heapq.heappop(queue)
11
12         if node in explored:
13             continue
14         explored.add(node)
15
16         x, y = node
17
18         if return_home:
19             if node == self.home_pos:
20                 self.route = path.copy()
21                 return
22         else:
23             if self.state.world[x][y] ==
24                 AGENT_STATE_UNKNOWN:
25                 self.route = path.copy()
26                 return
27
28         for dx, dy in [(1, 0), (0, 1), (-1, 0),
29             (0, -1)]:
30             nx, ny = x + dx, y + dy
31
32             # check boundary
33             if not (0 <= nx < self.state.
34                 world_width and 0 <= ny < self.
35                 state.world_height):
36                 continue
37             # check wall
38             if self.state.world[nx][ny] ==
39                 AGENT_STATE_WALL:
40                 continue
41
42             child = (nx, ny)
43             tentative_g_score = g_scores[node] +
44                 1
```

```
40         if child in explored or any(child ==
41             n for _, n, _ in queue):
42             continue
43
44         g_scores[child] = tentative_g_score
45         priority = tentative_g_score + self.
            heuristic(child, self.home_pos if
            return_home else (nx, ny))
        heapq.heappush(queue, (priority,
            child, path + [child]))
```

How it works The workflow is the same as Best-First Search. However, the logic to update the `queue` is quite different. Instead of just using $h(n)$ to calculate the `priority` score, we will define a new evaluation function called $f(n)$.

$$f(n) = g(n) + h(n)$$

$g(n)$ is the cost so far to reach n . Thus, our priority queue will be updated based on this $f(n)$

2 Comparison

Table 1: Comparison of Best-First Search and A* performance in different environment sizes.

Env. Size	Best-First Search			A*		
	Nodes	Len.	Optimal?	Nodes	Len.	Optimal?
	Explored		Score?	Explored		Score?
5 × 5	9	73	Optimal / 20	9	75	Non-optimal / 17
10 × 10	64	316	Optimal / 331	64	330	Non-optimal / 351
15 × 15	169	716	Non-optimal / 1093	169	713	Optimal / 692
20 × 20	324	1272	Optimal / 2153	324	1273	non-optimal / 2253

In this implementation, Best-First Search performance seems to be better than A*, but in practice, this is not always true. This is because Best-First Search only considers the heuristic function $h(n)$, allowing it to move quickly toward the goal and often expand fewer nodes. However, this greedy behavior can lead to suboptimal paths or getting misled in more complex problems.

In contrast, A* uses the evaluation function $f(n) = g(n) + h(n)$, which takes into account both the actual path cost and the heuristic estimate. Although A* may explore more nodes and require more computation, it is more reliable and guarantees an optimal solution when the heuristic is admissible.